

Nephtys: Lightweight Edge Connector for Bandwidth-Efficient Ingestion of Urban Sensor Streams

Andrea Bozzo
Independent Researcher
andreabozzo92@gmail.com

Abstract—Smart City sensor networks generate continuous telemetry that must be ingested, filtered, and forwarded by resource-conscious edge gateways. Existing tools span forwarding-oriented brokers and broader integration or stream-processing platforms, leaving different trade-offs in processing scope, reconfiguration semantics, and deployment footprint. We present Nephtys, a Go-based edge connector that ingests heterogeneous real-time sources (WebSocket, REST, SSE, webhooks, and gRPC) through hot-swappable per-stream middleware pipelines. NATS JetStream provides transport, event durability, and configuration storage without a separate application database. Three trials with 20 simulated stations reduced bytes by 67.30% and messages by 98.71%; five live feeds showed $88.96 \pm 0.34\%$ byte reduction. Against an equivalent Node-RED flow on identical event sequences, both systems produced identical reductions and near-identical latency, while Nephtys used 19.14 ± 0.07 MB versus 109.60 ± 0.40 MB tool RSS. On a Raspberry Pi 5 the same protocol reproduced these reductions and a $6.59\times$ memory advantage, though wall power was indistinguishable at ≈ 3.6 W, the platform’s idle floor dominating this workload.

Index Terms—Edge computing, IoT, Smart City, data ingestion, stream processing, NATS, middleware pipeline

I. INTRODUCTION

Smart City deployments and the Industrial Internet of Things (IIoT) have shifted data acquisition from static, batch-oriented collection to continuous streaming from ubiquitous sensor networks [1]. Urban air-quality stations, traffic sensors, and environmental probes can generate redundant telemetry across bandwidth-limited links between edge gateways and centralized backends. General-purpose distributed brokers and stream processors provide broad capabilities, but their operational model may be disproportionate for a gateway whose task is narrowly scoped to acquiring, reducing, and durably forwarding sensor streams [2], [3].

Forwarding mechanisms such as MQTT bridges [4], flow-based tools such as Node-RED [5], and configurable stream processors such as Redpanda Connect [6] cover overlapping parts of this task. Few lightweight designs combine multi-protocol acquisition, per-stream reduction, durable configuration without a separate database, and

replacement of processing logic while leaving the source connection running. We investigate this combination as a focused edge-connector design point rather than as a replacement for general IoT or stream-processing platforms.

This paper presents **Nephtys**, a telemetry ingestion connector developed in Go [7] and designed for resource-conscious edge deployment. It ingests WebSocket, REST-polling, Server-Sent Events, webhook, and gRPC sources. Each stream passes through a per-stream **middleware pipeline**: a composable chain of filter, transform, deduplicate, threshold, enrich, and batch stages that can be hot-swapped through a declarative REST API without restarting the source connector. Processed events are published to NATS JetStream [8]; initial stream registrations are stored in a JetStream Key-Value bucket to recover connectors after process restarts without a separate application database.

The contributions of this work are as follows:

- 1) A **multi-protocol ingestion architecture** with native fault tolerance (exponential-backoff reconnection, graceful shutdown) designed for intermittent edge connectivity.
- 2) A **dynamically reconfigurable middleware pipeline** engine that reduces upstream bandwidth by filtering, deduplicating, and compacting sensor payloads at the edge, evaluated on both synthetic and live urban air-quality streams.
- 3) A **database-free configuration persistence** model that uses the existing NATS JetStream subsystem for event durability and initial stream-registration state.

II. SYSTEM ARCHITECTURE AND RESILIENCE

Nephtys targets the constraints of edge computing: limited memory, unreliable network connectivity, and the absence of orchestration infrastructure [2]. Fig. 1 illustrates the overall architecture: urban sensor stations feed data into Nephtys through protocol-specific connectors; each stream traverses a configurable middleware pipeline before publication to NATS JetStream [8], from which cloud-side consumers subscribe.

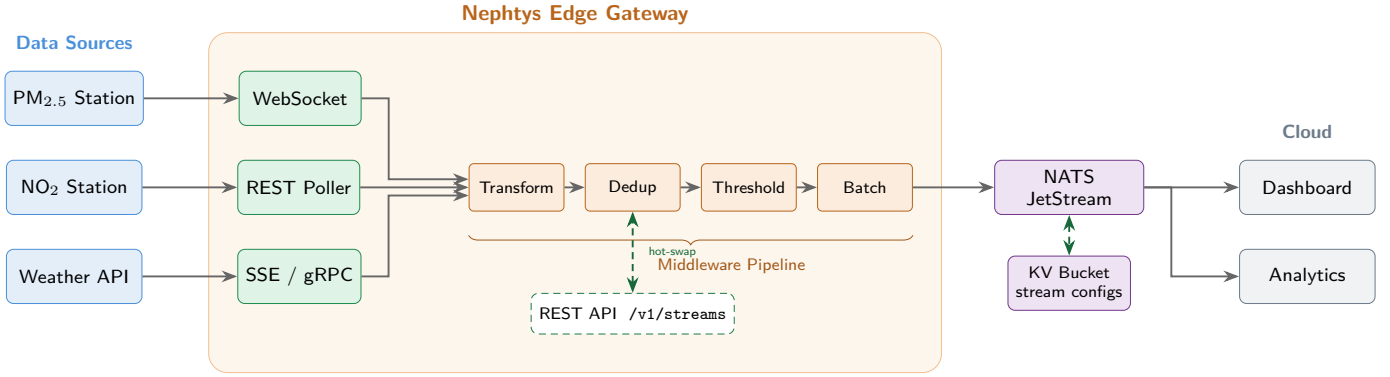


Fig. 1. Nephtys system architecture. Sensor data flows left-to-right through protocol connectors and a composable middleware pipeline before publication to NATS JetStream. The REST API enables runtime hot-swapping of pipeline configurations.

A. Fault-Tolerant Ingestion

Each data source is managed by a *connector* abstraction that encapsulates the protocol-specific logic (WebSocket, REST poller, SSE, webhook, gRPC). In a Smart City deployment [1], connectors may simultaneously ingest high-frequency air-quality readings via WebSocket from a local sensor gateway and poll a municipal open-data REST API for meteorological context.

The pull-based WebSocket and SSE connectors reconnect with **exponential backoff** (initial delay 1 s, capped at 30 s), limiting retry pressure when an upstream gateway is unavailable. REST polling retries on its next configured interval. Inbound webhook and gRPC sources instead delegate delivery retry to their upstream clients because they accept rather than initiate connections.

B. Database-Free Configuration Persistence

To avoid operating a separate application database, Nephtys uses **NATS JetStream** [8] as both event transport and configuration store. Published sensor events are retained in a durable JetStream stream (72 hours by default). Initial stream registrations are serialized in a JetStream Key-Value bucket (`nephtys_streams`); on startup, the Stream Manager reads these entries and re-registers their connectors and pipelines. Pipeline changes made through the hot-swap endpoint are intentionally transient in the evaluated version and revert to the registered configuration after restart.

This design keeps the required application stack to Nephtys and NATS; Prometheus and Grafana are optional evaluation tools. Outbound WebSocket and HTTP-based connectors accept TLS-protected endpoints, and the management REST API supports Bearer-token authorization. Single-board-computer behavior is measured directly on a Raspberry Pi 5 in Section III-B rather than assumed.

C. Edge Pipeline for Bandwidth Reduction

The central component is the **Pipeline Middleware** engine. Before an event reaches the broker, it traverses

a chain of composable middleware stages, each independently configurable per stream:

- **Transform:** Extracts relevant fields from nested JSON via dot-notation path mapping, reducing payload size.
- **Dedup:** Computes the FNV-64a hash [9] of each payload and blocks duplicates within a configurable time-to-live window.
- **Threshold:** Passes an event only if a numerical field changes by more than a configurable delta (e.g., $\Delta \text{PM}_{2.5} \geq 1.0 \mu\text{g}/\text{m}^3$), filtering sensor noise.
- **Batch:** Buffers events and flushes them as one aggregated message at a configurable interval or size, cutting per-message overhead.
- **Filter and Enrich:** Drop events not matching specified types, or inject static metadata tags.

The pipeline configuration is a JSON object attached to each stream registration and can be **hot-swapped at runtime** via a `PUT /v1/streams/{id}/pipeline` REST call. Internally, the active pipeline handler is stored behind a Go `atomic.Pointer`, so the swap is lock-free and takes effect on the next event with sub-millisecond latency. This allows operators to tighten or relax filtering thresholds in response to air-quality alerts without interrupting data flow.

III. EVALUATION SCENARIO

We evaluate Nephtys in an urban air-quality monitoring scenario representative of a Smart City edge deployment. All source code, the sensor simulator, and an automated benchmark script are publicly available¹ to allow full reproducibility of the results presented below.

A. Experimental Setup

Nephtys runs natively on Windows 11 (Intel Core Ultra 7 258V, 32 GB RAM); NATS JetStream, Prometheus,

¹<https://github.com/AndreaBozzo/uic2026-nephtys> (companion material); <https://github.com/AndreaBozzo/Nephtys> (Nephtys source).

and Grafana are co-deployed via Docker Compose. The evaluated Nephtys revision is `c146ee7`. Two classes of data sources feed the system:

- 1) **Synthetic streams.** A lightweight WebSocket-based simulator (`sensor-sim`) generates air-quality readings for 20 virtual stations at 2 Hz, producing ~ 40 events/s. Each event carries $\text{PM}_{2.5}$, PM_{10} , NO_2 , temperature, and humidity values with configurable jitter and a 30% duplicate ratio, modeling the redundancy typical of low-cost particulate-matter sensors.
- 2) **Real-world open data.** Five REST poller streams ingest live measurements from the Open-Meteo weather and air-quality APIs [10] for three cities in Calabria (Cosenza, Catanzaro, Reggio Calabria) and from a Sensor.Community [11] citizen PM sensor near Gioia Tauro, polled every 30–60 seconds.

The full middleware pipeline is configured as: Transform (5 fields extracted), Dedup (TTL = 30 s, cache = 500), Threshold (path = `pm25`, $\delta = 1.0 \mu\text{g}/\text{m}^3$), Batch (size = 50, flush = 5 s). A controlled comparison uses Node-RED 5.0.1 natively on the same host with core WebSocket, JSON, Change, RBE, and Join nodes plus a thin NATS JavaScript client wrapper. Its immediate RBE and global deadband match the observed semantics of the pinned Nephtys revision for this workload. A neutral subscriber verifies the timestamp-independent retained-event sequence and measures normalized output bytes and generation-to-publication latency. Tool RSS is Windows working set; CPU is normalized so 100% denotes one logical CPU. Each system processes the same 12,000-event sequence after a discarded 1,200-event warm-up, in interleaved order over three trials.

Edge-hardware platform. The identical protocol is repeated on a Raspberry Pi 5 (4 GB, active cooler, default governor, no overclock) running both tools and NATS natively on 64-bit Raspberry Pi OS, with the simulator and collector on a separate wired host. RSS and CPU come from `/proc`; wall power is measured at the mains socket, so it covers the whole board and its official supply, with no idle baseline subtracted. As the meter accumulates energy in 0.206 Wh blocks, energy integrates its instantaneous power reading at ~ 1.4 s intervals, agreeing with the meter’s own register within one block over the session.

B. Results

Table I reports three trials of two experiments: (a) a 5-minute synthetic workload with 20 virtual stations at 2 Hz (~ 40 events/s, 30% duplicate ratio), and (b) a 30-minute live workload with five REST poller streams hitting the Open-Meteo [10] and Sensor.Community [11] APIs. The simulator uses a fixed seed. Each table value is the arithmetic mean; reduction ratios and RSS additionally show sample standard deviation. Live inputs vary with the upstream APIs.

Synthetic workload. Each controlled trial ingested exactly 12,000 events; 7,733 survived Dedup and Thresh-

TABLE I
BENCHMARK RESULTS OVER THREE TRIALS (MEAN; $\pm$ SAMPLE SD).

Metric	Baseline	Pipeline	Reduction
<i>Synthetic streams (20 stations, 5 min)</i>			
Bytes published	2,730,398	892,955	67.30 $\pm$ 0.00%
Messages published	12,000	155	98.71 $\pm$ 0.00%
<i>Real-data streams (5 APIs, 30 min)</i>			
Bytes published	142,489	15,729	88.96 $\pm$ 0.34%
Messages published	270	20	92.71 $\pm$ 0.22%

TABLE II
CONTROLLED COMPARISON ON BOTH PLATFORMS (MEAN $\pm$ SAMPLE SD; THREE TRIALS EACH).

Metric	x86-64 host		Raspberry Pi 5	
	Nephtys	Node-RED	Nephtys	Node-RED
Byte reduction (%)	67.30 $\pm$ 0.00	67.30 $\pm$ 0.00	67.30 $\pm$ 0.00	67.30 $\pm$ 0.00
Message reduction (%)	98.71 $\pm$ 0.00	98.71 $\pm$ 0.00	98.71 $\pm$ 0.00	98.71 $\pm$ 0.00
Tool RSS (MB)	19.14 $\pm$ 0.07	109.60 $\pm$ 0.40	19.51 $\pm$ 0.07	128.47 $\pm$ 0.44
Tool + NATS RSS (MB)	27.15 $\pm$ 0.09	117.31 $\pm$ 0.36	38.85 $\pm$ 0.10	147.07 $\pm$ 0.48
CPU (% of one core)	0.03 $\pm$ 0.02	0.31 $\pm$ 0.08	0.32 $\pm$ 0.00	0.72 $\pm$ 0.01
Latency p50 (ms)	1004.33 $\pm$ 0.58	1006.33 $\pm$ 0.58	1006.00 $\pm$ 0.00	1009.00 $\pm$ 1.00
Latency p95 (ms)	2004.33 $\pm$ 0.58	2006.67 $\pm$ 0.58	2009.00 $\pm$ 1.00	2013.00 $\pm$ 1.00
Wall power (W)	—	—	3.610 $\pm$ 0.005	3.584 $\pm$ 0.014
Energy per event (mJ)	—	—	92.2 $\pm$ 0.3	91.5 $\pm$ 0.4

old and were compacted into 155 batches. Transform and filtering reduced published bytes by 67.30% and message count by 98.71%. The retained-event sequence was identical across systems in every paired trial.

Real-data workload. Across the five live streams, each 30-minute trial ingested 270 events on average; Dedup dropped 158 and Threshold 92, leaving 20 published messages, reducing bytes by 88.96% and messages by 92.71%. One trial saw a Sensor.Community connection reset that recovered at the next poll, illustrating live-input variability.

Resilience. When the simulator process is killed, the WebSocket connector detects the disconnection and enters exponential backoff (1 s, 2 s, 4 s, ..., capped at 30 s), then reconnects within 1–2 s of the simulator restarting. Ingestion pauses and resumes with no crash or manual intervention.

Hot-reload latency. A PUT changing the threshold delta completes in under 16 ms including the HTTP round-trip, and the atomic pointer swap takes effect on the next event with no loss or reordering.

Table II shows that the equivalent flows have identical reduction and similar latency, dominated by batching. Nephtys uses 82.5% less tool RSS and 76.9% less complete-stack RSS on the host. CPU is low for both at ~ 40 events/s, so no saturation claim is made.

Edge hardware. On the Pi all six slots were valid on the first attempt, the SoC never throttled (`throttled=0x0` across 1,316 samples, 45–51°C), and every slot reproduced the host’s retained-event sequence hash and its 155 output batches, so the pipelines remain equivalent across architectures. The memory advantage widens to $6.59\times$ tool RSS and $3.79\times$ for the complete stack: Node-

RED costs 17.2% more resident memory on ARM64, while Nephtys is essentially unchanged at +1.9%. Wall power, however, is *indistinguishable*, at 3.610 ± 0.005 W against 3.584 ± 0.014 W — a 0.70% difference, of the opposite sign to the memory result and below one quantisation step of the meter’s reading. The board draws ≈ 3.0 W idle, so at 40 events/s the platform floor dominates and the footprint advantage does not become measurable energy savings. We therefore claim no energy advantage in either direction: on this hardware the smaller footprint buys headroom for co-located workloads, not power.

IV. RELATED WORK AND CONCLUSIONS

A. Related Work

Table III compares capabilities separately from the controlled Node-RED measurement. EdgeX Foundry [12] is a broad microservice-based IoT platform, while MQTT bridges [4] focus on broker-to-broker forwarding. Node-RED [5] provides general flow-based processing and persists deployed flows to files. Redpanda Connect is the closest configuration-driven comparator: its streams mode exposes runtime CRUD for isolated processor pipelines, but an update shuts down and replaces the affected stream [6]. Nephtys deliberately offers fewer processors and sinks; its narrower distinction is swapping a processing chain while retaining the running source connector, coupled with JetStream-backed registration state.

B. Conclusions

This paper presented Nephtys, a lightweight edge connector that demonstrates how compiled languages and lightweight message brokers can enable bandwidth-efficient telemetry ingestion for Smart City sensor networks. By shifting deduplication, transformation, and threshold filtering to the edge through declaratively configurable middleware pipelines [3], the system reduced bytes by 67.30% on the controlled synthetic workload and 88.96% on the live workload. An equivalent Node-RED flow confirmed identical reduction and latency within 3 ms, while Nephtys used 19.14 MB mean tool RSS versus 109.60 MB, and 19.51 MB versus 128.47 MB on a Raspberry Pi 5. That run also bounds the benefit: wall power there was indistinguishable, so the advantage is memory headroom rather than energy.

The runtime-reconfigurable pipeline allows operators to adapt filtering behavior (e.g., tightening thresholds during pollution alerts) without restarting the source connector. The contribution is the evaluated combination of this

update boundary, heterogeneous acquisition, and reuse of the NATS durability layer—not a claim that runtime-reconfigurable stream processing is unique to Nephtys.

Current limitations include only three trials, a 30-minute live-data window with changing upstream values, one quantitative platform baseline, and a single-node deployment. The pinned revision applies threshold state globally rather than per station. The evaluated system also uses at-least-once delivery, and pipeline hot-swaps are not persisted across restart. The edge measurement covers a single load point on one board, and its power figures come from an uncalibrated consumer meter, so they characterize the platform rather than resolve small differences between tools. These constraints limit statistical and platform-level generalization of the reported results.

Immediate future work is broader controlled baselines and an event-rate sweep on the edge board, since the point at which processing cost overtakes the idle power floor is precisely what a single load point cannot reveal. Longer-term work will investigate adaptive pipeline policies that trade information loss and processing latency against bandwidth and resource budgets, using multi-node sensor deployments only after the single-node operating envelope is characterized.

REFERENCES

- [1] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, “Internet of things for smart cities,” *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, 2014.
- [2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [3] M. D. de Assunção, A. da Silva Veith, and R. Buyya, “Distributed data stream processing and edge computing: A survey on resource elasticity and future directions,” *Journal of Network and Computer Applications*, vol. 103, pp. 1–17, 2018.
- [4] A. Banks, E. Briggs, K. Borgendale, and R. Gupta, “MQTT version 5.0,” OASIS, OASIS Standard, 2019, 07 March 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>
- [5] OpenJS Foundation, “Node-RED – low-code programming for event-driven applications,” <https://nodered.org>, 2026, accessed: 2026-03-26.
- [6] Redpanda Data, “Redpanda Connect streams api,” https://docs.redpanda.com/connect/guides/streams_mode/streams_api/, 2026, accessed: 2026-07-13.
- [7] The Go Authors, “The Go programming language specification,” <https://go.dev/ref/spec>, 2026, accessed: 2026-03-26.
- [8] NATS Authors, “NATS – the cloud native messaging system,” <https://nats.io>, 2026, accessed: 2026-03-26.
- [9] G. Fowler, L. C. Noll, K.-P. Vo, D. E. Eastlake, III, and T. Hansen, “The FNV non-cryptographic hash algorithm,” IETF Internet-Draft, <https://datatracker.ietf.org/doc/html/draft-eastlake-fnv-17>, 2019, internet-Draft (expired).
- [10] P. Zippenfenig, “Open-Meteo – free weather API,” <https://open-meteo.com>, 2026, accessed: 2026-03-26.
- [11] Sensor.Community, “Sensor.Community – a contributors driven global sensor network,” <https://sensor.community>, 2026, accessed: 2026-03-26.
- [12] LF Edge, “EdgeX Foundry – open-source edge computing platform for IoT,” <https://www.edgexfoundry.org>, 2026, accessed: 2026-03-26.

TABLE III

QUALITATIVE COMPARISON OF OVERLAPPING EDGE-INGESTION TOOLS.

System	Primary scope	Runtime change	Config state
MQTT bridge	Broker forwarding	Reconnect/reload	Broker-specific
Node-RED	General visual flows	Flow redeploy	Local flow file
EdgeX	IoT microservices	Service/rule APIs	Service databases
Redpanda Connect	General pipelines	Stream replacement	Files; runtime API
Nephtys	Source-to-NATS edge	Pipeline-only swap	KV registration